SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

NAME: M.S.Charishma

REG.NO:192311393

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I _M.S.Charishma__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.S.Charishma*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANSWERS

1. Online Banking Fund Transfer Testing

The failures at minimum and maximum transfer limits indicate inadequate boundary testing. Equivalence Partitioning (ECP) should divide transfer amounts into invalid-below-minimum, valid, and invalid-above-maximum classes. Boundary Value Analysis (BVA) should test values just below, exactly at, and just above both limits. For example, for an allowed range of ₹100–₹50,000, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001, along with representative middle values. Random testing can discover unexpected defects, but it may miss rare boundary conditions and should supplement, not replace, systematic testing. Prioritize boundary tests, valid/invalid classes, error messages, transaction integrity, and regression tests.

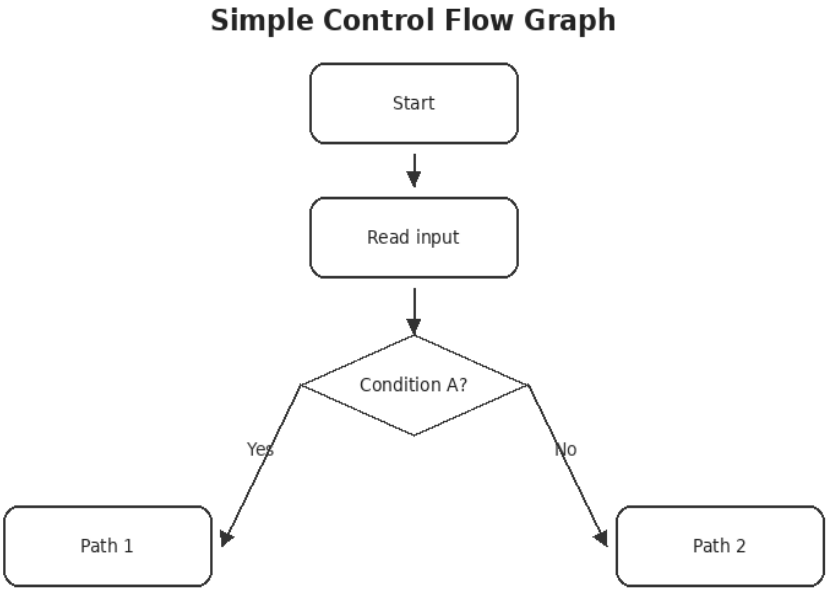
2. Healthcare Application: White-Box Coverage

Post-deployment defects suggest that the test suite did not adequately cover decision outcomes and execution paths. Construct a control flow graph (CFG) with nodes for input validation, authentication, patient-data validation, processing, error handling, and output. Test both outcomes of every decision, including valid and invalid inputs, authorized and unauthorized access, and normal and exception paths. Statement coverage only confirms that statements execute; branch coverage checks decision outcomes; path coverage checks execution-path combinations but may be impractical for complex systems. Use statement and branch coverage targets, basis-path testing, risk-based tests, boundary tests, and requirements traceability. Validate safety-critical and privacy-related paths, then perform regression testing to improve reliability and reduce patient-safety risks.

3. Retail E-Commerce Input Combinations

Testing price, quantity, and discount separately misses defects caused by interactions between fields. Apply ECP to classify price and quantity as valid positive values or invalid zero/negative values, and discount as valid, zero, or outside its permitted range. Use BVA around minimum and maximum price, quantity, and discount limits. Decision-table testing should combine conditions such as valid price, valid quantity, discount eligibility, and maximum discount to verify the expected total for each meaningful rule. Include cases for zero quantity, negative price, 100% or excessive discount, rounding, and invalid combinations. Exhaustive testing may be too large, so use a decision table and pairwise combinations for lower-risk interactions, while testing all high-risk business rules. This improves coverage without unnecessary duplicate tests.

4. Complex Nested Conditions and Loops



Partial path coverage may leave important combinations of nested decisions and loop behaviour untested. First draw a CFG showing each statement, decision, loop entry, loop exit, and return path. Calculate cyclomatic complexity using V(G) = E − N + 2P, where E is the number of edges, N is the number of nodes, and P is the number of connected components; for one connected component, V(G) = E − N + 2. Select a basis set of linearly independent paths and create tests that execute each path. For loops, test zero iterations, one iteration, typical iterations, and values at or near the maximum permitted count. Check both outcomes of nested conditions and loop termination. Use risk-based prioritization where exhaustive path testing is infeasible, and measure branch and basis-path coverage to identify remaining gaps.

5. Financial Application: Hybrid Testing

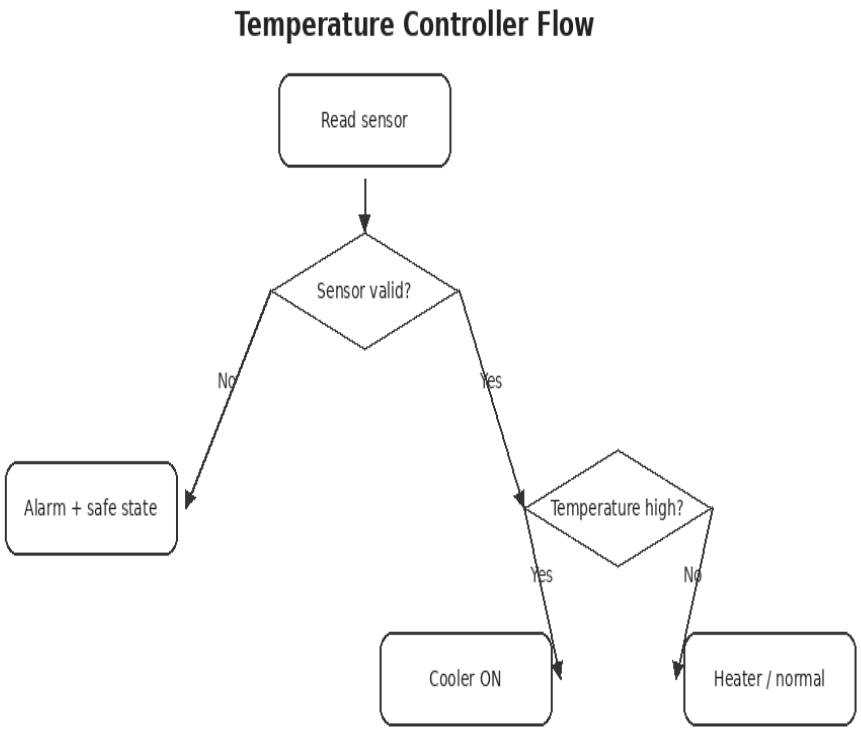
Defects in production show that black-box and white-box testing may have been performed without sufficient traceability, coverage review, or integration. Black-box ECP and BVA should test valid and invalid account values, transaction amounts, limits, fees, and rounding boundaries without relying on implementation details. White-box testing should use a CFG to identify decision branches, exception paths, authorization checks, and transaction commit or rollback paths. Design tests from requirements and map each test to the relevant code and risk. Measure statement and branch coverage, and use basis-path testing for critical transaction logic. Add integration, security, concurrency, and regression tests, including interrupted transactions and duplicate requests. Review coverage gaps before release; coverage metrics support confidence but do not prove the absence of defects.

6. HR Payroll Bonus Boundary Testing

Incorrect payouts at exactly 5, 10, and 15 years suggest that threshold comparisons or service-band rules may be implemented incorrectly. Confirm the policy for each threshold and test one value below, exactly at, and one value above it—for example, 4 years 11 months, 5 years, and 5 years 1

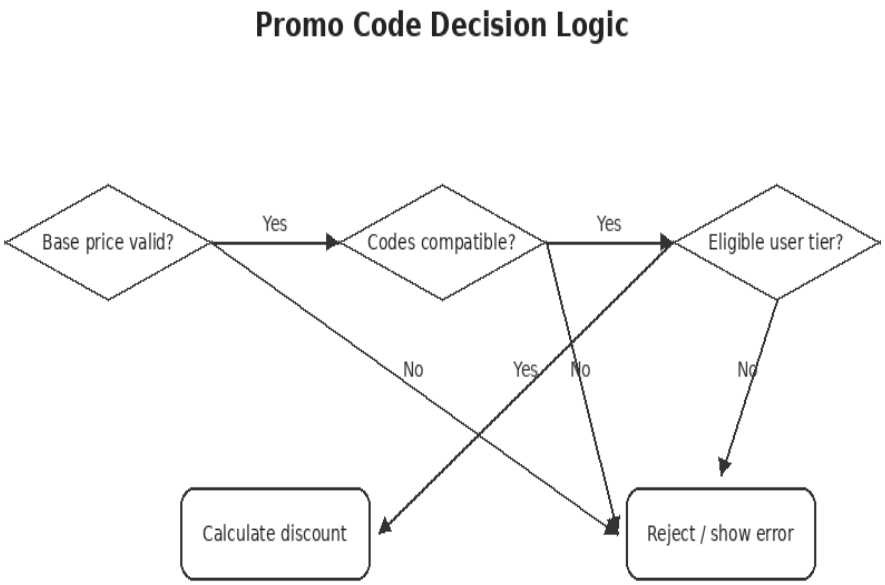
month, with equivalent cases around 10 and 15 years. ECP should classify employees into each service band and performance-rating category, including invalid ratings and missing service data. Test combinations of service band and rating, and verify the calculated bonus against independently calculated expected results. BVA is especially effective because the reported failures occur at exact thresholds. Exploratory testing can complement the suite but is less repeatable and may overlook these precise values. Regression-test all bonus rules after correction.

7. Laboratory Temperature Controller and MC/DC



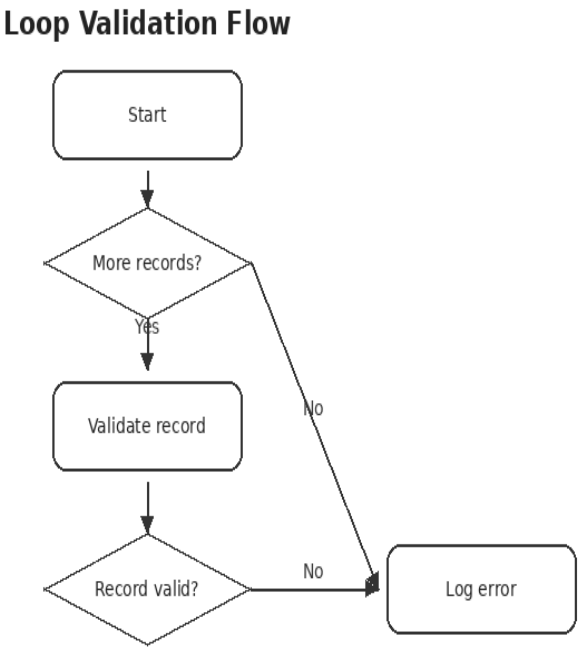
Represent the control logic in a CFG with nodes for reading the sensor, checking sensor validity, comparing temperature with upper and lower limits, controlling the heater or cooler, and triggering an alarm. The reported failure may arise because the sensor-fault condition was not combined with other decision conditions in tests. Identify each Boolean condition in every safety-related decision and design Modified Condition/Decision Coverage (MC/DC) tests showing that each condition can independently change the decision outcome while other conditions are held appropriately constant. Include valid sensor readings, readings above and below limits, sensor disconnection, out-of-range values, and fault sequences. Verify that an invalid sensor triggers a safe state and alarm. Achieve and document 100% MC/DC for the identified logic where required by the safety standard, alongside branch coverage, hardware-in-the-loop tests, and regression tests.

8. Travel Booking Promo-Code Combinations



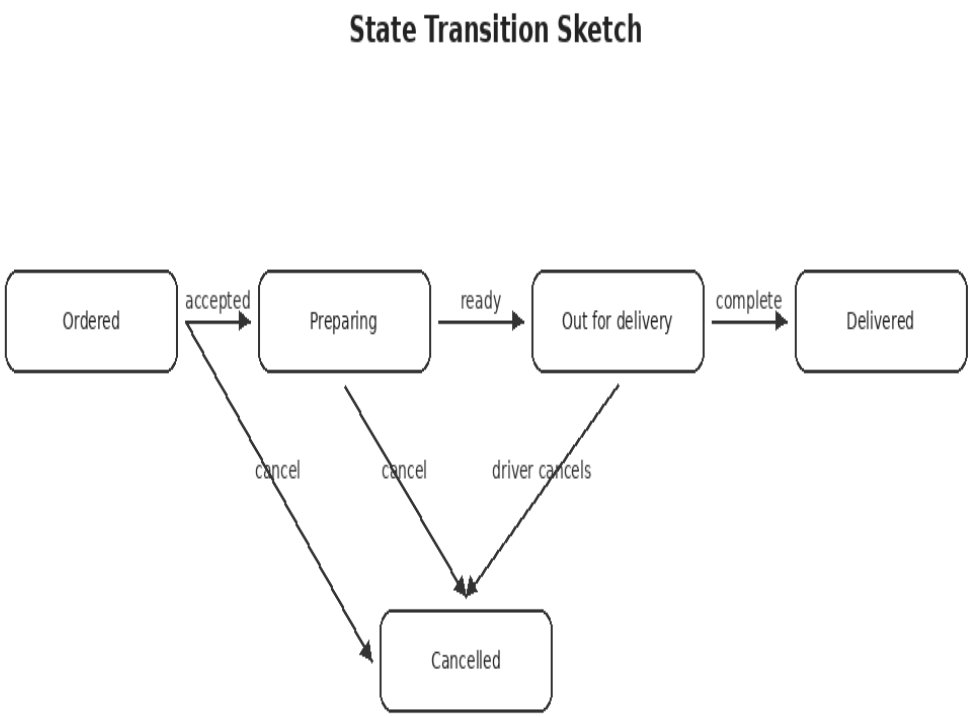
Testing each promo code independently cannot expose interactions that produce a negative total. Create a decision table using conditions such as base price validity, promo-code validity, code compatibility, user tier, minimum-spend eligibility, and discount limits. Define actions for accepting or rejecting each code, applying eligible discounts in the correct order, and preventing the total from falling below zero. Test valid single codes, compatible codes, mutually exclusive codes, expired codes, ineligible user tiers, discounts equal to the price, and discounts greater than the price. Include boundary values for minimum spend and discount caps. Use rule-based tests for every distinct business outcome and pairwise combinations for lower-risk interactions to reduce redundancy. Assert that the final total is never negative and that the system explains rejected codes clearly.

9. Database Migration Tool: Loops and Cyclomatic Complexity



High statement coverage only proves that statements were executed; it does not show that loop exit conditions, repeated iterations, or combinations of nested decisions were tested. Draw a CFG for the nested loops and validation branches, then calculate cyclomatic complexity using V(G) = E − N + 2P. Use the result to estimate the number of independent basis paths that need tests. For each loop, test zero records, one record, multiple records, the maximum allowed batch, empty or malformed records, and conditions that should terminate processing. Verify that loop counters and exit conditions always progress and that errors do not cause endless retries. Include nested-loop combinations, timeouts, cancellation, and recovery tests. Monitor execution time and termination, and use code review and regression testing to address infinite-loop risks.

10. Food Delivery Real-Time Tracker



State Transition Testing is appropriate because the tracker changes state over time and may fail when cancellation occurs mid-delivery. Define states such as Ordered, Preparing, Out for Delivery, Delivered, and Cancelled, with permitted events and transitions. Test the normal sequence from Ordered to Delivered, cancellation before preparation, cancellation during preparation, driver cancellation while Out for Delivery, and invalid transitions such as moving from Delivered back to Preparing. Check that the interface, timestamps, notifications, refund or reassignment rules, and stored status remain consistent. ECP and BVA focus mainly on input values and may miss defects caused by event order and state history. Build a transition table, cover every valid transition and important invalid transition, and test repeated or delayed events to prevent the tracker from becoming stuck.

11. Cybersecurity Authentication: Hybrid Testing

Brute-force vulnerabilities may indicate missing rate limits, lockout controls, OTP expiry checks, or incomplete testing of authentication decision paths. Use black-box tests to try repeated wrong passwords, expired and reused OTPs, invalid input formats, suspicious IP locations, and attempts after lockout. Error Guessing should target common weaknesses such as unlimited retries, predictable responses, timing differences, and account enumeration. For white-box testing, draw a CFG of password verification, OTP validation, IP-risk checks, retry counters, and allow/deny decisions; test independent paths and both outcomes of each condition. Verify rate limiting, safe lockout and recovery, audit logging, and consistent error messages. Combine automated security tests with code review and regression testing, and ensure that security controls do not reveal sensitive account information.

12. E-Learning Subscription Expiry and Upgrade

The failure on the exact expiry date suggests that date boundaries and upgrade rules were not fully tested. Use ECP to classify upgrade attempts as before expiry, on the expiry date, after expiry, and outside the valid subscription period. Test Free, Basic, and Premium plans, including allowed and disallowed course types and payment status. Apply BVA to timestamps immediately before, at, and after expiry, while confirming the system's time zone and whether expiry is inclusive or exclusive. Test upgrades, renewals, failed payments, duplicate requests, and access rights after a plan change. Use a decision table to combine date class, subscription tier, course eligibility, and payment result. Verify that the correct access is granted without a gap or unauthorized extension, and run regression tests on related subscription rules.

13. Smart-Home Lighting: Branch and Condition Coverage

Statement coverage is insufficient because a statement may execute while one Boolean outcome or switch-case branch remains untested. Build a CFG for sensor reading, threshold comparisons, switch cases, dimming levels, and sensor-error handling. Branch coverage requires tests for both true and false outcomes of decisions and each relevant switch case; condition coverage requires each Boolean subcondition to evaluate to both true and false. Test ambient light below, at, and above thresholds, combinations of sensor status and light level, invalid readings, and default switch cases. Where expressions are complex, use MC/DC for important decisions. Check the actual light output as well as software values through hardware-in-the-loop testing. This approach exposes logic and integration faults that statement-only testing may miss and improves dependable operation.

14. Shipping Cost and Invalid Weight Boundaries

The missed zero and negative weights indicate that input validation did not cover invalid boundaries. If the minimum permitted weight is greater than zero, test a negative value, zero, the smallest valid value, a value just above the minimum, and values at and above the maximum. Use ECP to classify weights as invalid negative, invalid zero if prohibited, valid, and over-limit; also test invalid text and missing values. Combine representative weight classes with destination zones and delivery speeds to verify the shipping-rate rules. The application should reject invalid weights with a clear message and should not calculate or store an incorrect charge. Robust BVA is essential because invalid values often expose comparison and validation errors. Add regression tests to ensure validation works consistently across all shipping options.

15. Medical Imaging: Prime Data Sizes and Basis Path Testing

Random testing may miss failures that occur specifically for prime-sized datasets because it does not guarantee that those sizes or relevant execution paths are exercised. Construct a CFG for dataset validation, partitioning, parallel-loop execution, synchronization, and result aggregation. Calculate cyclomatic complexity and select a basis set of independent paths. Test small and large datasets, prime and composite sizes, sizes around chunk or thread boundaries, empty input, and incomplete final partitions. Compare parallel outputs against a trusted sequential implementation and check repeatability across runs. Inspect loop bounds, remainder handling, synchronization, and race conditions. Basis Path Testing provides systematic control-flow coverage, while targeted prime-

size tests address the observed pattern. Add stress, concurrency, and regression tests to support mathematical accuracy and stability.